

Instruction for Reproducing the Reported Results

COMP0197 Group 17

Uncertainty-Aware DC-LSTM for FX Forecasting

1. Overview

This submission contains a standalone codebase for training and evaluating several LSTM-based forecasting models for GBP-USD foreign exchange prediction with uncertainty estimation.

The codebase includes the following main components:

- **train.py**: downloads the dataset, performs feature engineering and feature selection, prepares sequential inputs, trains all models, and saves the final trained weights.
- **test.py**: loads the saved models and produces the final evaluation metrics, tables, and visual results.
- **src/**: supporting modules for preprocessing, model definitions, training, evaluation, and plotting.
- Saved model files (**.pt**): final trained weights for all reported models.

2. Python Environment

The project was developed in Python 3.12.

The code uses the following main libraries:

- PyTorch
- NumPy
- pandas
- matplotlib
- scikit-learn

2.1. Additional Installed Packages

The following additional packages were used in this project and may need to be installed if they are not already present in the Python environment:

- pandas
- matplotlib
- scikit-learn

Installation command:

```
pip install pandas matplotlib scikit-learn
```

3. Folder Structure

Please keep the submission folder structure unchanged. The main files and folders expected by the code are:

- `train.py`
- `test.py`
- `src/`
- `data/raw/`
- `data/interim/`
- `data/processed/`
- saved model files in the project root
- output folders generated during execution, such as:
 - `training_figures/`
 - `evaluation_results/`

4. Important Execution Order

Please run the scripts in the following order:

1. Run `train.py`
2. Run `test.py`

This order is required because `train.py` downloads the raw dataset, creates intermediate processed files, and saves the trained model weights that are later loaded by `test.py`.

5. How to Run the Code

Open a terminal in the project root directory.

5.1. Step 1: Train All Models

Run:

```
python train.py
```

This script will:

1. Download the dataset from the GitHub raw URL.
2. Save the raw data locally in `data/raw/`.
3. Perform data cleaning, stationary transformation, feature engineering, and feature selection.

4. Save the selected intermediate dataset to `data/interim/interim_selected.csv`.
5. Prepare scaled sequence data.
6. Train all reported models:
 - Baseline LSTM
 - Gaussian LSTM
 - Dual-Branch LSTM
 - Gaussian Dual-Branch LSTM
 - DC-LSTM
 - Gaussian DC-LSTM
 - BiLSTM
 - Gaussian BiLSTM
7. Save the final trained model weights as `.pt` files in the project root.
8. Save training figures in the `training_figures/` folder.

5.2. Step 2: Evaluate All Models

After training is complete, run:

```
python test.py
```

This script will:

1. Load the saved model checkpoints.
2. Rebuild the test sequences from the saved intermediate dataset.
3. Evaluate deterministic, MC Dropout, Gaussian, and MC+Gaussian predictions.
4. Generate final metrics, performance summaries, price-based evaluation tables, and plots.
5. Save all final outputs under the `evaluation_results/` folder.

6. Main Output Files

After running both scripts, the main outputs will include:

6.1. Training Outputs

- Intermediate selected dataset:
 - `data/interim/interim_selected.csv`
- Feature metadata:
 - `data/interim/selected_features_metadata.json`
- Training figures:
 - `training_figures/`

6.2. Saved Model Files

The training script saves the final trained weights for all models, including files such as:

- `best_baseline_model.pt`
- `best_gaussian_model.pt`
- `best_dual_branch_model.pt`
- `best_gaussian_dual_model.pt`
- `best_dc_lstm_model.pt`
- `best_gaussian_dc_lstm_model.pt`
- `best_bilstm_model.pt`
- `best_gaussian_bilstm_model.pt`

6.3. Evaluation Outputs

The testing script saves outputs under `evaluation_results/`, including:

- performance summary tables
- benchmark comparison tables
- price metrics tables
- cumulative return and performance plots
- uncertainty-aware price path plots

7. Notes for Reproduction

1. The scripts assume execution from the project root directory.
2. `test.py` should only be run after `train.py` has completed successfully.
3. The dataset is downloaded automatically by `train.py`, so no manual dataset download is required.
4. Reproducibility is supported through fixed random seeds in both training and testing scripts.
5. If a GPU is available, PyTorch may use it automatically; otherwise the code runs on CPU.

8. Summary

To reproduce all reported results:

1. Open a terminal in the project root.
2. Run `python train.py`.
3. Wait until all model checkpoints are saved.
4. Run `python test.py`.
5. Check the generated figures, tables, and saved results in the output folders.

9. Pipeline Dependency Between Training and Testing

`train.py` produces the intermediate data files and trained model checkpoints used by `test.py`. For this reason, the two scripts are intended to be used together as one reproducible pipeline.